

Postgres Performance Report — 4 findings (1 DB / 1 app / 2 mixed) — 2026-06-16

1 message

gydamohaimeed@gmail.com <gydamohaimeed@gmail.com>16 June 2026 at 15:37

To: gydamohaimeed@gmail.com

Postgres Performance Report

Generated 2026-06-16 — all numbers measured from pg\_stat\_statements and EXPLAIN ANALYZE, never estimated by the LLM.

4TOTAL FINDINGS

1DATABASE

1APPLICATION

2MIXED

1VERIFIED

Summary

| # | Pathology        | Layer | Confidence | Trend | Query                                                           | Calls | Mean ms | Total ms | Status                                   |
|---|------------------|-------|------------|-------|-----------------------------------------------------------------|-------|---------|----------|------------------------------------------|
| 1 | DEEP_OFFSET      | MIXED | HIGH       |       | SELECT id, status, total_cents FROM orders ORDER BY created_... | 60    | 123.92  | 7435.21  | ✔ Verified — measured improvement        |
| 2 | UNBOUNDED_RESULT | APP   | HIGH       |       | SELECT id, customer_id, status FROM orders WHERE id::text LI... | 50    | 58.03   | 2901.41  | Not applicable — fix is application-side |
| 3 | UNBOUNDED_RESULT | MIXED | HIGH       |       | SELECT oi.id, oi.order_id, oi.product_id FROM order_items oi... | 10    | 95.91   | 959.12   | Not verified                             |
| 4 | LEADING_WILDCARD | DB    | LOW        |       | SELECT id, email FROM customers WHERE email LIKE \$1            | 100   | 8.01    | 800.68   | Not verified                             |

Fix in the database (1)

DDL or configuration changes, verifiable by the agent.

LEADING\_WILDCARD    LOW    calls 100 · mean 8.01 ms · total 800.68 ms · rows/call 100000.00

```
SELECT id, email FROM customers WHERE email LIKE $1
```

**Root cause:** The customers table has no index on the email column. Even if a btree index existed, a leading-wildcard LIKE pattern (e.g. LIKE '%example.com') cannot use a standard btree index and forces a full sequential scan across all 100,000 rows. The pg\_stat\_statements rows\_per\_call of 100,000 indicates that virtually every row in the table is being returned per call, suggesting either no leading wildcard in production (in which case a btree index would help) or that the pattern is effectively unfiltered. To support both suffix and arbitrary substring searches efficiently, a pg\_trgm GIN index on email is the appropriate fix, as it supports all LIKE patterns including those with leading wildcards.

**Evidence:** The plan shows a Seq Scan on customers (100,000 rows) with Filter '(email ~~ \$1::text)', removing 99,999 rows to return 1. No index on the email column exists (only a btree PK on id). The pg\_stat\_statements rows\_per\_call=100,000 with only 100 calls and 8ms mean suggests the sampled parameter used for EXPLAIN was not a leading wildcard (it returned 1 row), but production calls returning 100,000 rows/call strongly indicate either no leading wildcard in production or an UNBOUNDED\_RESULT compounded by a missing index. The \$1 parameter is

normalized, so the actual production value cannot be confirmed. The btree index that would normally serve a suffix LIKE cannot be used for a leading-wildcard pattern. Confidence is LOW because the sampled parameter may not represent the real production value.

**Proposed fix:** CREATE EXTENSION IF NOT EXISTS pg\_trgm; CREATE INDEX idx\_customers\_email\_trgm ON public.customers USING gin (email gin\_trgm\_ops);

**Tradeoffs:** GIN index is larger than a btree index and slower to update on INSERT/UPDATE; trigram index requires pg\_trgm extension; for patterns longer than ~3 characters performance is excellent but very short patterns may still fall back to seq scan; if leading wildcards are not actually used in production a plain btree index on email would be smaller and faster for equality and prefix searches

**Not verified**

## Fix in your code (1)

ORM / application changes — the database is doing what it was asked.

UNBOUNDED\_RESULT HIGH calls 50 · mean 58.03 ms · total 2901.41 ms · rows/call 111111.00

```
SELECT id, customer_id, status FROM orders WHERE id::text LIKE $1
```

**Root cause:** The application issues `SELECT ... FROM orders WHERE id::text LIKE \$1` with no LIMIT, and each call returns ~111,111 rows — roughly 5.5% of the 2-million-row orders table. At this selectivity a sequential scan is correct and no index will help. The root problem is the application retrieving unbounded, massive result sets: either the LIKE pattern is too broad (e.g. a short prefix or a leading wildcard), or the caller genuinely needs all matching rows but has no pagination strategy. Fetching 111k rows per call is wasteful regardless of the pattern type.

**Evidence:** rows\_per\_call=111111 on a 2M row table (5.5% of table per call). HypoPG confirmed that a functional index on (id::text) text\_pattern\_ops yields 1.00x speedup — the planner is correct to seq scan at this selectivity. No index can help when ~5.5% of rows are returned. The query has no LIMIT clause. The pattern \$1 on id::text is returning enormous result sets each call (50 calls × 111k rows = 5.5M rows total). Total time 2901ms across 50 calls, 58ms mean with parallel seq scan.

**Proposed fix:** Add pagination to the query: apply a LIMIT (e.g. 100–1000 rows) and use keyset pagination on the `id` column rather than OFFSET. If the use-case requires searching orders by a string representation of their ID, consider exposing a more selective search (e.g. require at least 8 characters of prefix) or storing a pre-cast text column with tighter constraints. If the full result set is genuinely needed, move the operation to a background batch job rather than a synchronous query.

**Tradeoffs:** Pagination changes the API contract and requires client-side changes; keyset pagination requires a stable sort key; restricting pattern length may require UI changes; background batching adds latency for the consumer

**Not applicable — fix is application-side**

## Fix in both layers (2)

Findings that need a database change and an application change.

DEEP\_OFFSET HIGH calls 60 · mean 123.92 ms · total 7435.21 ms · rows/call 20.00

```
SELECT id, status, total_cents FROM orders ORDER BY created_at LIMIT $1 OFFSET $2
```

**Root cause:** The query implements classic OFFSET-based pagination over a 2-million-row table sorted by created\_at. Because no index exists on created\_at, PostgreSQL cannot use an index scan to cheaply skip to the desired page — instead it performs a full parallel sequential scan of the entire orders table (~148 MB read from disk: 18,547 shared read blocks), sorts all rows using an external merge sort that spills to disk, and only then applies the LIMIT/OFFSET. As the OFFSET grows larger (later pages), the wasted work grows proportionally. The fix requires two coordinated changes: (1) a database-side index on created\_at so the planner can use an Index Scan with efficient row skipping, and (2) an application-side switch from OFFSET pagination to keyset (cursor) pagination using the last seen created\_at and id values, which eliminates deep-offset wasted work entirely and makes each page fetch O(1) regardless of position.

**Evidence:** The plan shows a full sequential scan of ~2,000,000 rows (Seq Scan actual rows=666,667 × 3 workers = ~2M), followed by parallel external-merge sorts spilling to disk (Temp Read Blocks=13,806, Temp Written Blocks=24,306), all to return just 50 rows. Gather Merge materializes 100,050 rows before the Limit node discards nearly all of them. Mean execution time is 123.9ms with a 318ms execution time in the sampled plan. The sort is on created\_at but no index exists on that column (only orders\_pkey on id exists). With \$2 as an OFFSET parameter, large offset values force the engine to scan and sort the entire table every time. The n\_distinct=1.0 statistic for created\_at is suspicious and may indicate stale/misleading stats, but the primary driver is the OFFSET-based pagination pattern combined with a missing index on the sort key.

**Proposed fix:** DB layer — create the index: `CREATE INDEX CONCURRENTLY orders_created_at_id_idx ON public.orders (created_at, id)`; APP layer — replace OFFSET pagination with keyset pagination: Instead of `'ORDER BY created_at LIMIT $1 OFFSET $2'`, use: `SELECT id, status, total_cents FROM orders WHERE (created_at, id) > ($last_created_at, $last_id) ORDER BY created_at, id LIMIT $page_size`; The application must track the (created\_at, id) of the last row returned and pass it as the cursor for the next page.

**Tradeoffs:** CONCURRENTLY index build avoids table lock but takes longer and uses extra I/O; composite (created\_at, id) index supports both sort and tie-breaking but adds ~30-50 MB index storage; keyset pagination cannot support random page jumps (must navigate sequentially), requires a stable sort key, and complicates 'jump to page N' UI features; adding id as a tie-breaker requires the query and index to agree on sort order; existing queries sorting only by created\_at without id will benefit from the index but may still need id for determinism

✓ **Verified — measured improvement**

HypoPG estimate: plan cost 144294.36 → 5631.85 (25.62× estimated)

Measured: 176.07 ms → 10.08 ms (delta 165.99 ms, 17.47× speedup)

UNBOUNDED\_RESULT HIGH calls 10 · mean 95.91 ms · total 959.12 ms · rows/call 120252.00

```
SELECT oi.id, oi.order_id, oi.product_id FROM order_items oi JOIN orders o ON o.id = oi.order_id WHERE o.status = $1
```

**Root cause:** Two compounding problems exist. At the database layer, there is no index on orders.status, forcing a full sequential scan of the 2M-row orders table (148 MB) on every call. However, because status has only 4 distinct values, a plain btree index would have very low selectivity (~90% of rows match each status) and the planner would likely still prefer a Seq Scan for large result fractions — so the index benefit is limited. The deeper, structural problem is at the application layer: the query returns an average of 120,252 rows per call with no LIMIT or pagination, meaning the application is pulling hundreds of thousands of rows into memory on every invocation. This is the root pathology. The hash join also spills to disk (15,846 temp blocks read and written), compounding the I/O cost. The query needs to be redesigned with server-side pagination or streaming, and if only a subset of columns is needed downstream, a narrower query should be issued.

**Evidence:** rows\_per\_call=120252 with no LIMIT clause; orders.status has n\_distinct=4 (low cardinality), so each status value matches ~90% of the 2M rows (actual: 1,799,338 of 2,000,000 orders matched). The plan does full Seq Scans on both order\_items (2M rows) and orders (2M rows), produces 1,799,387 rows per call, spills to disk (Temp Read/Written Blocks: 15,846 each for the hash join), and runs for ~1 second per call. No index on orders.status exists. The result set is massive and unbounded — the application is fetching over 1.8M rows per query.

**Proposed fix:** Application fix (primary): Introduce keyset/cursor-based pagination — e.g. `'SELECT oi.id, oi.order_id, oi.product_id FROM order_items oi JOIN orders o ON o.id = oi.order_id WHERE o.status = $1 AND oi.id > $2 ORDER BY oi.id LIMIT 1000'` — so the application processes results in bounded batches rather than fetching 120K+ rows at once. Database fix (secondary, to support paginated queries): `CREATE INDEX orders_status_idx ON public.orders USING btree (status)`; `CREATE INDEX order_items_order_id_idx ON public.order_items USING btree (order_id)`; — these indexes become useful once the query is paginated and selectivity improves per batch.

**Tradeoffs:** Pagination adds application-side loop complexity; keyset pagination requires a stable sort key; the status index has low selectivity for dominant status values so may not be used for those; batching increases total round-trips but dramatically reduces per-call memory and I/O pressure; work\_mem could be raised to avoid spill but would not address the fundamental unbounded result problem

**Not verified**

Application-side findings are reported with evidence but marked unverified — code changes cannot be benchmarked by this agent.